# Introducción a los lenguajes de programación

Rafael López García
(rafalg@cifprodolfoucha.es)
CIFP Rodolfo Ucha Piñeiro

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL
Plan de Recuperación, Transformación y Resiliencia
Financiado por la Unión Europea
NextGenerationEU
XUNTA DE GALICIA
CENTRO INTEGRADO DE FORMACIÓN PROFESIONAL RODOLFO UCHA PIÑEIRO
IES Valle del Jerte PLASENCIA
CENTRO DE EXCELENCIA DE FP CIBERSEGURIDAD

# Parte I: Introducción a los lenguajes de programación

# Definición

- Un **lenguaje de programación** es un lenguaje con unas reglas bien definidas que permiten elaborar un programa
- Un **programa** es una secuencia de instrucciones que permiten controlar el comportamiento físico o lógico de un sistema informático

# Lenguaje máquina (I)

- El **lenguaje máquina** es el único lenguaje que entiende un procesador de un ordenador
  - Se trata de una secuencia de números binarios que representan las instrucciones del programa
    - P.ej.: 100011 00011 01000 00000 00001 000100
  - Cada instrucción es tan larga como marca la arquitectura del procesador
    - Por lo general, 32 o 64 bits

# Lenguaje máquina (II)

- Características
  - Muy difícil de entender para humanos, pues es muy **antinatural**
  - Necesita muchas instrucciones para hacer tareas aparentemente simples
  - Completamente **dependiente del procesador** para el que se programa

# Lenguaje ensamblador (I)

- El **ensamblador** emplea mnemotécnicas para facilitar la escritura de instrucciones
  - P.ej.: MOV AL, 61h
    - Carga en el registro AL el número 61 hexadecimal
- Más fácil de entender que el lenguaje máquina pero:
  - Muy **dependiente del procesador** empleado
  - Es **difícil elaborar programas complejos** con él
- Todavía se usa para programar drivers, etc.

# Lenguaje ensamblador (II)

```
000 ; Alignment     : default
000
000                 include uni.inc ; see unicode subdir of ida for info on unicode
000
000                 .686p
000                 .mmx
000                 .model flat
000
000 ; ===========================================================================
000
000 ; Segment type: Pure code
000 ; Segment permissions: Read/Execute
000 _text           segment para public 'CODE' use32
000                 assume cs:_text
000                 ;org 401000h
000                 assume es:nothing, ss:nothing, ds:_data, fs:nothing, gs:nothing
000
000 ; =============== S U B R O U T I N E =======================================
000
000
000                 public start
000 start           proc near
000                 mov     eax, TlsIndex
005                 shl     eax, 2
008                 mov     dword_40B0DB, eax
00D                 push    edi
00E                 push    ecx
00F                 xor     eax, eax
011                 mov     edi, offset CriticalSection
016                 mov     ecx, offset unk_40D848
01B                 cmp     ecx, edi
01D                 jbe     short loc_401024
01F                 sub     ecx, edi
021                 cld
022                 rep stosb
024
024 loc_401024:                             ; CODE XREF: start+1D↑j
024                 pop     ecx
025                 pop     edi
026                 push    0
028                 call    sub_40161B
02D                 pop     ecx
02E                 push    offset off_40B0A0
```

# Lenguajes de alto nivel (I)

- Los **lenguajes de alto nivel** son más cercanos a los humanos (generalmente combinan palabras en inglés con expresiones matemáticas)

```
long factorial (int n)  {
  if (n == 0) {
    return 1;
  } else {
    return(n * factorial(n-1));
  }
}
```

# Lenguajes de alto nivel (II)

- Además, los lenguajes de alto nivel suelen ser más **independientes del procesador**
  - En muchas ocasiones el mismo programa podría funcionar en Windows de 32 bits, Windows de 64 bits, en Linux, en MacOS, etc.
  - En otras ocasiones, se necesitan ligeras modificaciones para adaptarlo

# Lenguajes de alto nivel (III)

- Una instrucción en código ensamblador suele equivaler a una instrucción en código máquina
  - Pasar de una a otra es trivial
- Sin embargo, **las instrucciones de los lenguajes de alto nivel son muy complejas** en comparación con las del ensamblador
  - 20 instrucciones sencillas en lenguaje C pueden equivaler a cientos o miles en ensamblador

# Problema

- Como hemos dicho, el ordenador sólo entiende lenguaje máquina
- Pero el humano generalmente escribe en un lenguaje de alto nivel
  - Más rápido y sencillo elaborar programas complejos
- **Es necesario traducir el lenguaje de alto nivel a lenguaje máquina** antes de poder ejecutar las instrucciones

# Lenguajes compilados (I)

- Un **lenguaje compilado** es aquel que requiere que todas las instrucciones sean traducidas a lenguaje máquina antes de ser ejecutadas
- A dicho proceso de traducción se le llama **compilación** y al programa que la realiza se le llama **compilador**
- Aunque en realidad internamente se usan 3 programas uno tras otro: el compilador, el ensamblador y el enlazador

# Lenguajes compilados (II)

- El resultado de llamar al compilador suele ser un fichero ejecutable para un cierto SO y arquitectura de procesador
  - Ejecutable Win PE (.exe, .dll, etc.) para Windows
  - Ejecutable ELF para Linux
  - Ejecutable Mach-O para MacOS e iOS
  - Un programa para un procesador de 64 bits no funcionará en uno de 32 bits

# Lenguajes compilados (III)

Fichero holamundo.c

```
#include <stdio.h>
int main() {
   // printf() displays the string inside quotation
   printf("Hello, World!");
   return 0;
}
```

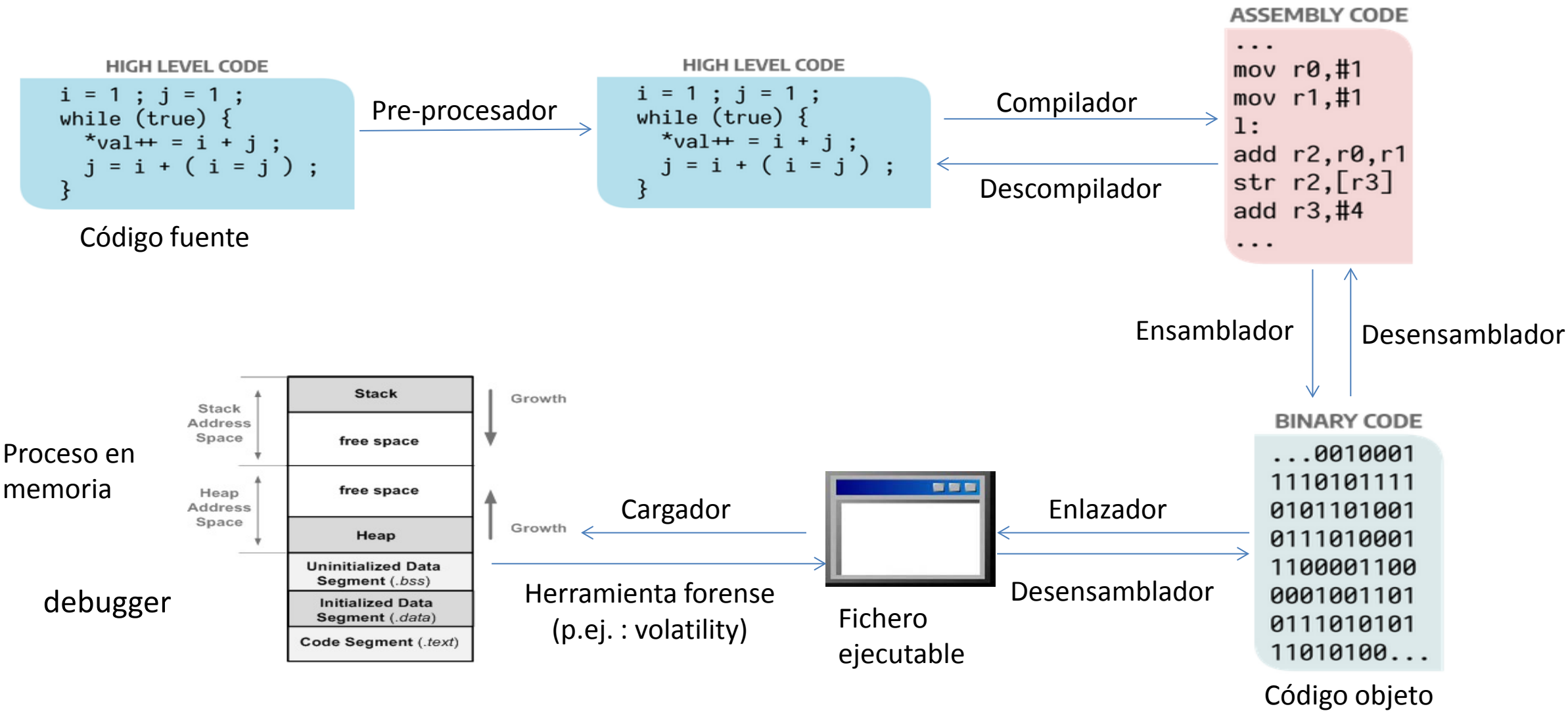
COMPILACIÓN

$ gcc -o holamundo holamundo.c ⟶ Genera el programa ejecutable “holamundo”

EJECUCIÓN

$ ./holamundo ⟶ Ejecuta el programa “holamundo”

# Lenguajes compilados (IV)



# Lenguajes interpretados

- Los **lenguajes interpretados** van traduciendo las instrucciones a lenguaje máquina una a una antes de ser ejecutadas
- Al programa que realiza este proceso se le llama **intérprete**
- Típico de los lenguajes de script de los SO
  - Bash, PowerShell, etc.
- Pero también hay otros lenguajes
  - Ruby, Python, JavaScript, etc.

# Lenguajes compilados vs lenguajes interpretados

- Lenguajes compilados
  - La compilación tarda tiempo, pero la ejecución es más rápida
  - Se necesita crear un ejecutable para cada plataforma
  - Durante la compilación se previenen fallos de sintaxis y se optimiza el código
  - **Mayor carga para el programador, menor carga para la máquina**
- Lenguajes interpretados
  - No requieren traducción previa, pero la ejecución es más lenta
  - El ciclo de desarrollo más rápido
  - Más fácil cometer fallos de sintaxis
  - **Menor carga para el programador, mayor carga para la máquina**

# Lenguajes parcialmente compilados (I)

- Los creadores de Java tenían como objetivo desarrollar un lenguaje de programación que funcionara en cualquier SO y arquitectura
- La estrategia de Java se basa en compilar a un lenguaje intermedio cercano al lenguaje máquina llamado *bytecode*
- Este lenguaje será interpretado después por la máquina virtual Java

# Lenguajes parcialmente compilados (II)

- Para ejecutar un programa en Java se necesita que la máquina tenga instalado el **JRE o Java Runtime Environment**
- Para compilar un programa en Java se necesita que la máquina tenga instalado el **JDK o Java Development Kit**
- Otros lenguajes que siguieron esta aproximación son C#, Elixir, etc.

# Parte II: Paradigmas de programación

# Paradigmas de programación

- Programación imperativa
  - El programador instruye a la máquina cómo cambiar su estado
- Programación declarativa
  - el programador simplemente declara las propiedades del resultado deseado, pero no cómo calcularlo
- Multiparadigma

# Paradigmas de programación

- **Programación imperativa**
  - **El programador instruye a la máquina cómo cambiar su estado**
- Programación declarativa
  - el programador simplemente declara las propiedades del resultado deseado, pero no cómo calcularlo
- Multiparadigma

# Programación imperativa

- El programador instruye a la máquina cómo cambiar su estado
- Subtipos:
  - Estructurada
  - Procedimental
  - Modular
  - Orientada a Objetos

# Programación estructurada, procedimental y modular (I)

- Estructurada: el flujo de control se define mediante bucles anidados, condicionales y subrutinas
- Procedimental: las instrucciones se engloban en procedimientos o funciones y éstas se llaman cada vez que haya que ejecutar la instrucciones
- Modular: dividir un programa en módulos o subprogramas con el fin de hacerlo más manejable y legible

# Programación estructurada, procedimental y modular (II)

- Ejemplos
  - PASCAL
  - C
  - BASIC
  - COBOL
  - FORTRAN
  - Bash

```c
void bubblesort(int array[],int size)
{
    int tmp ,i,j;
    for(i = 0;i <size;i++)
        for(j=0;j < size;j++)
            if(array[i] < array[j])
            {
                tmp = array[i];
                array[i] = array[j];
                array[j] = tmp;
            }
}
```

Programa en lenguaje C

```
 GNU nano 4.8                                  bash.sh
#!/bin/bash
read -p "Enter number: " x
if [ $x -eq 5 ]
   then
      echo "Number matched.. End of for loop!"
      exit 0
fi
echo "Number doesn't match..."
```

Programa en bash

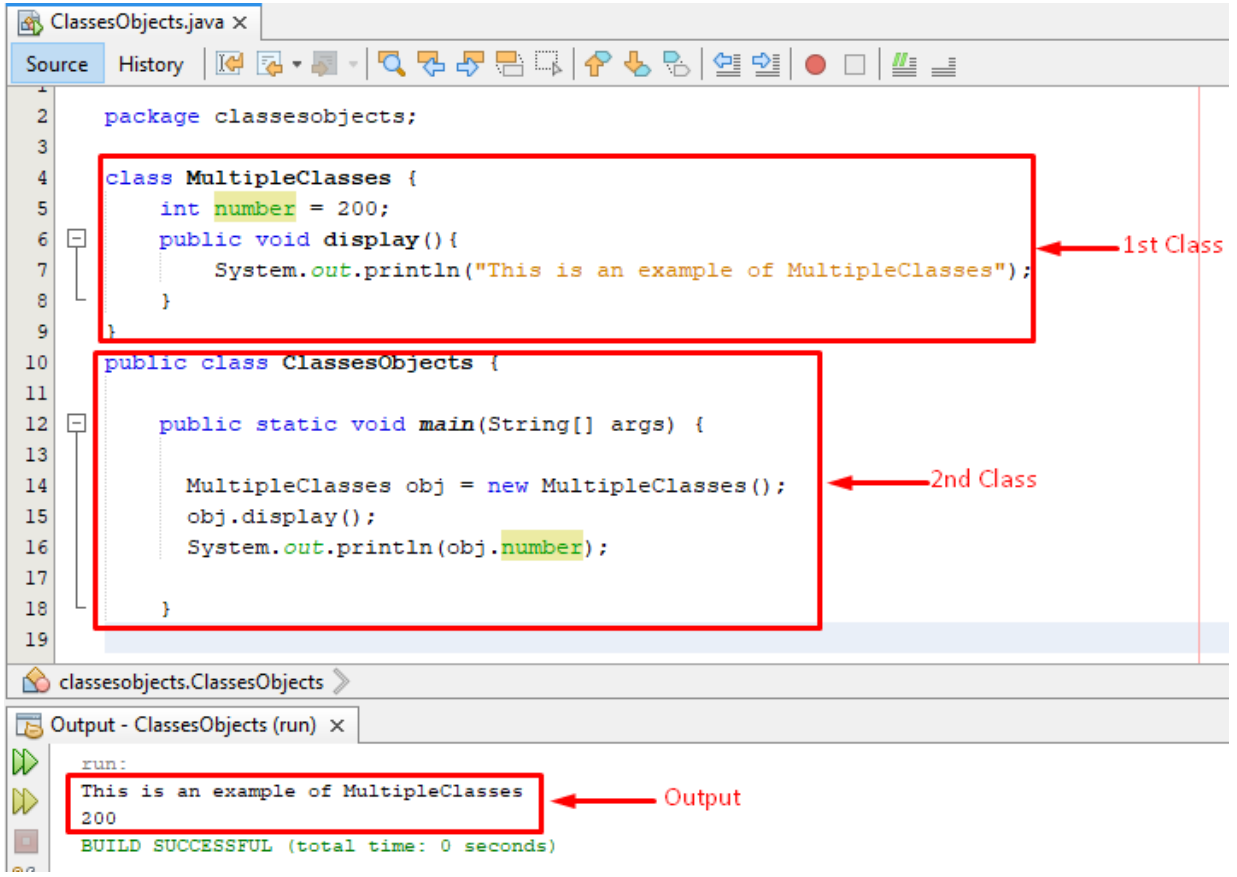
# Programación orientada a objetos (I)

- En la programación orientada a objetos (POO) se definen los conceptos de clase y objeto, que abstraen y en capsulan el código
  - Maximiza la cohesión y minimiza el acoplamiento
  - La herencia entre clases permite un mayor nivel de reutilización y extensibilidad del código
  - El polimorfismo permite mandar mensajes a objetos de distintas subclases como si fueran de su superclase

# Programación orientada a objetos (II)

- Ejemplos
  - Visual Basic.NET
  - SmallTalk
  - JavaScript
  - Java
  - C++
  - C#
  - PHP
  - Fortran 90/95

Clases en Java



# Paradigmas de programación

- Programación imperativa
  - El programador instruye a la máquina cómo cambiar su estado
- **Programación declarativa**
  - **el programador simplemente declara las propiedades del resultado deseado, pero no cómo calcularlo**
- Multiparadigma

# Programación declarativa

- El programador simplemente declara las propiedades del resultado deseado, pero no cómo calcularlo
- Subtipos
  - Funcional
  - Lógica
  - Otros (matemática, reactiva, etc.)

# Programación funcional

- El resultado deseado se declara como el valor de una serie de aplicaciones de función
- Ejemplos:
  - LISP
  - Erlang
  - Scala
  - Haskell
  - Caml
  - SQL

Código en Caml

```
# let rec sum l =
    match l with
    | [] -> 0                    (* base case *)
    | hd :: tl -> hd + sum tl    (* inductive case *)
  ;;
val sum : int list -> int = <fun>
# sum [1;2;3];;
- : int = 6
```

# Programación lógica

- El resultado deseado se declara como la respuesta a una pregunta sobre un sistema de hechos y reglas
- P. ej.: Prolog

```
/* sintaxis: suma(X,Y,Z). */
/* semántica: X + Y = Z */
suma(0,X,X).
suma(s(X),Y,s(Z)):-suma(X,Y,Z).

/* sintaxis: producto(X,Y,Z). */
/* semántica: X * Y = Z */
producto(0,X,0).
producto(s(0),X,X).
producto(s(X),Y,G):-producto(X,Y,Z),suma(Z,Y,G).
```

```
?- producto (0,3,1).
False

?- producto (4,3,12).
True
```

# Multiparadigma

- Hoy en día los lenguajes han introducido **elementos de diversos paradigmas**, con lo cual es más difícil clasificarlos sólo en uno
  - P.ej.: Java 8 introdujo los *lambdas*, que son programación funcional
  - P.ej.: Scala es funcional y orientado a objetos
  - P.ej.: Ocaml introdujo orientación a objetos a Caml
  - P.ej.: Python soporta programación imperativa, orientada a objetos y, en menor medida, funcional